Shell Script
& Makefile & Git

SK UNIVERSITY

Interface Between User & OS

Shell

What is Shell?

- **Interface program between user & UNIX**
  - Similar with Windows command prompt
  - Bash version: `$ bash --version`
- **Shell takes role of**
  - Control commands (Unix)
  - Advanced programming language

Redirection & Pipes

- **File descriptors:**
  - descriptors used to access files in Unix systems
  - standard file descriptors
    - **`stdin`** (0): keyboard input
    - **`stdout`** (1): terminal output
    - **`stderr`** (2): terminal output for error messages
- **Redirection: redirects standard I/O to files**
  - `<` : redirect **`stdin`**
  - `fd>` : redirect **`fd`** (overwrites existing content)
    - Default: stdout (1) (i.e., `echo How are you? > text.txt`)
    - Creates file if it does not exist
  - `fd>>` : same as `fd>`, but appends instead
- **Pipe**
  - Connects process output to another process input
  - *e.g.*, `$ cat test.txt | grep "a"`

Redirection & Pipes

```
$ ls -al > output.txt
$ ps >> output.txt
$ cat output.txt | sed 's/^/> /'
```

```
> total 8
> drwxrwxr-x  2 user user 4096 Mar 17 10:53 .
> drwxrwxr-x 16 user user 4096 Mar 17 10:53 ..
> -rw-rw-r--  1 user user    0 Mar 17 10:54 output.txt
>     PID TTY          TIME CMD
>  332019 pts/1    00:00:00 bash
>  333996 pts/1    00:00:00 ps
```

Shell Programming

- **Two ways of shell programming**
  - Line command
  - Execute a shell script

```
$ ls
$ mv a b
$ cp b b_copy
$ rm b
```

```
$ cat test.sh
#!/usr/bin/env bash
ls
mv a b
cp b b_copy
rm b

$ bash test.sh
$ sh test.sh
$ ./test.sh
```

You can specify interpreter (even different shell)
In shell scripts, **#** is for comments.

Executing scripts

- **First line of scripts (shebang line)**
  - The path to the *interpreter* that will process the text file
  - ***#!****(interpreter-path)* i.e., **`#!/usr/bin/env bash`**
- **Any Unix command can go in a shell script**
  - Commands are executed in order or
    in the flow determined by control statements
  - Shells have their own dialects for control structures
- **To make a script executable,**
  - **`$ chmod u+x [filename]`**

```
$ chmod u+x test.sh
$ ./test.sh
```

Grammars of Shell

- **Variable**
  - String, number, environment, parameter
- **Condition**
  - Boolean
- **Condition control**
  - if, elif, for, while, until, case
- **List**
- **Function**
- **Other reserved commands (other keywords)**

Shell Variable

- **Declaration = Initialization of variable**
- **Dereference shell variable by "$"**
  - Do not confuse this with the "$" to represent the start of line command.
- **Can check value of variable by "echo"**
- **No space between variable name and value!**

```
$ VAR="Hi“
$ echo $VAR
Hi
$ VAR="How are you?“
$ echo $VAR
How are you?
```

More on $

- **Aside from dereferencing variables, …**
- **$ can be used to print command output**
- **$ can expand arithmetic expressions.**
  - This allows you to dynamically calculate with bash using expressions in double parentheses like ((expression)) .
  - Can pass variable names without dereferencing with $.
  - Arithmetic expressions can be used for conditionals!

```
$ echo "I am $(whoami) and I am in $(pwd)"
I am spl and I am in /home/spl
$ echo $(( $(date +%s) / 60 / 60 / 24 / 365 ))
years have passed since 1970
55 years have passed since 1970
$ A=1; B=1; echo $((A+B))
2
```

Shell Predefined Variables

- **Some shell variables are initialized**
- **Environment variables**
  - shell variables used to define execution environment
  - uses upper case letters (check with **env** command)
  - Existing variables can be checked with **env** command
- **Users can set their own environment**

Predefined Variable	Description
$HOME	Home directory of current user
$PATH	Directories for search command, each separated by “:”
$LD_LIBRARY_PATH	Directories for search library, each separated by “:”
$0	Name of shell script (bash by default)
$#	Number of passed parameter
$$	Process ID of shell script

Shell Parameter Variable

- **If arguments were passed to the shell script,**
  - We can access the arguments with **`$1,$2,...`**

```
$ cat test.sh
#!/usr/bin/env bash
echo $0
echo $1
echo $2
echo $3

$ ./test.sh Shell Script Test
./test.sh
Shell
Script
Test
```

Shell Conditional Statement

- **`if`**
  - Checks condition and executes command block
  - Also we can use **`else`** & **`elif`** (else if)

```
if condition
then
  statement
  …
elif condition
then
  statement
  …
else
  statement
  …
fi
```

```
#!/usr/bin/env bash
read num
if (( $num < 5 )); then
    echo "Lower than 5"
elif (( $num > 8 )); then
    echo "Greater than 8"
else
    echo "5~8"
fi
```

Conditions

- **File test**
  - **`-e`**: Returns true if file exists
  - **`-d`**: Returns true if file exists and is a directory
  - Usage: **`if [[ -e file.txt ]]`**
- **String test: `=, !=, <, >`**
  - Usage: **`if [[ “a” != “b” ]]`**
- **Arithmetic test**
  - Arithmetic expressions, wrapped with double parentheses, can be used as conditionals
  - Similar syntax to C (**`<, >, ==, >=, <=, ||, &&`**)
  - Usage: **`if (( a == b || a >= 3))`**

For loop

- **`for`**
  - Iterates for range of values
  - Range of values can be a set of strings

```
for variable in values
do
    statements
done
```

```
#!/usr/bin/env bash
for x in {0..2} a b c; do
    echo $x
done
```

```
0
1
2
a
b
c
```

```
for ((init; cond; update;))
do
    statements
done
```

```
#!/usr/bin/env bash
for ((i=0;i<=3;i++))
do
    echo $i
done
```

```
0
1
2
```

While loop

- **`while`**
  - Iterates while condition is true

```
while condition
do
    statements
done
```

```
x=1
while [ $x -le 12 ]; do
    echo $x
    x=$(($x+1))
done
```

Can be written as a for-loop

```
for x in {1..12}; do
    echo $x
done
```

Simple Way for Compiling

Makefile

Why Makefile?

- Simplifies compiling source codes
- Describes the relationships among files
- Provides commands for updating each file
- Recompiles each changed file

```
gcc -o test main.c test.c hello.c
```

vs.

```
make --file=Makefile
```

Or just `make`

Rule

- **Structure of rules in a "*Makefile*"**

```
target ...: prerequisites ...
    recipe
    ...
```

  - A *rule* explains how and when to remake certain files, or to carry out an action
  - A *target* is the name of a file that is generated by a program, or the name of an action to carry out (ex. clean, install)
  - A *prerequisite (dependency)* is a file that is used as in put to create the target
  - A *recipe* is an action that 'make' carries out

How make process

- **Recursive rule processing**
  - processes prerequisites based on creation times of files

```
def process(target = first_rule) # default target is first rule
  return true if target.phony?
  return target.modified? unless target.rule?

  is_modified = target.prerequisites.any? do |prerequisite|
    process prerequisite # recursive processing
  end

  execute target.recipe if is_modified
  is_modified
end
```

How make process

- **If 'main.c' has been modified**
  - *make* proceeds as follows

```
# Makefile
hello: main.o hello.o
    gcc -Wall -o hello main.o hello.o
main.o: main.c
    gcc -Wall -c main.c
hello.o: hello.c
    gcc -Wall -c hello.c
clean:
    rm *.o hello
```

(1) Find default goal
recursively process prerequisites

(5) execute recipes for target

(2) process 1st prerequisite

(3) execute recipes for 1st prerequisite
main.c is modified (newer than main.o)

(4) process 2nd prerequisite
hello.c is not modified (older than hello.o)
skip recipes for 2nd prerequisite

Variable

- **Variables in Makefile**
  - Defined once and substituted in multiple places
  - Substitute variable's value by writing **`$(variable)`**

```
# Makefile
TARGET=hello
CC=gcc
CFLAGS=-Wall
OBJECTS=main.o hello.o
$(TARGET): $(OBJECTS)
    $(CC) $(CFLAGS) -o $(TARGET) $(OBJECTS)
main.o: main.c
    $(CC) $(CFLAGS) -c main.c
hello.o: hello.c
    $(CC) $(CFLAGS) -c hello.c
clean:
    rm $(OBJECTS) $(TARGET)
```

Automatic variables

- **Variables computed by executing rules**
  - **$@** : name of the target of the rule
  - **$^** : names of all the prerequisite of the rule
  - **$?** : names of all the prerequisites that are newer than the target
  - **$<** : name of first prerequisite

```
$(TARGET): $(OBJECTS)
    $(CC) $(CFLAGS) -o $@ $^
main.o: main.c
    $(CC) $(CFLAGS) -c $< -o $@
hello.o: hello.c
    $(CC) $(CFLAGS) -c $< -o $@
```

Rules

- **Phony targets**: When the target is not an actual file, the target can be made a prerequisite for **`.PHONY`**.
- **Pattern rules**: `%` is used as a wildcard to match any string in a pattern (e.g. `%.c` can refer to any C file).
- **Implicit rules**: Not all targets require a recipe as some rules are built-in in **`make`**. like `%.c -> %.o` .

```
$(TARGET): $(OBJECTS)

# %.o: %.c
#   $(CC) $(CFLAGS) -c $< -o $@

.PHONY: clean

clean:
    rm -f $(OBJECTS) $(TARGET)
```

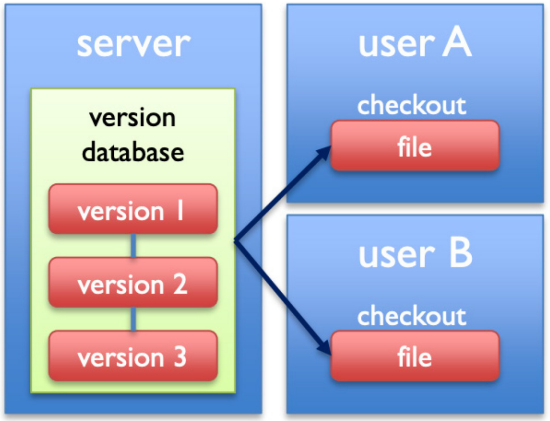
Version Control System

Git

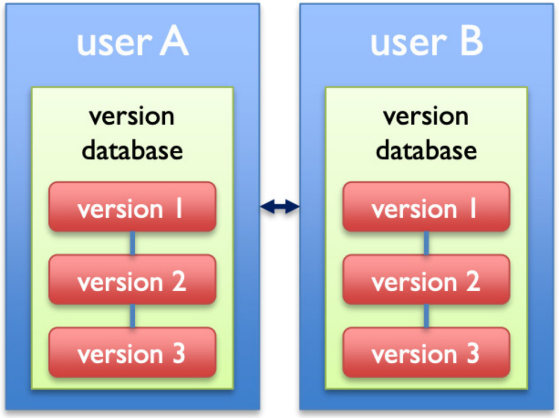
Version Control System

- **Version Control System (VCS)**
  - Manages changes of documents, computer programs, and other collections of information

Centralized revision control system



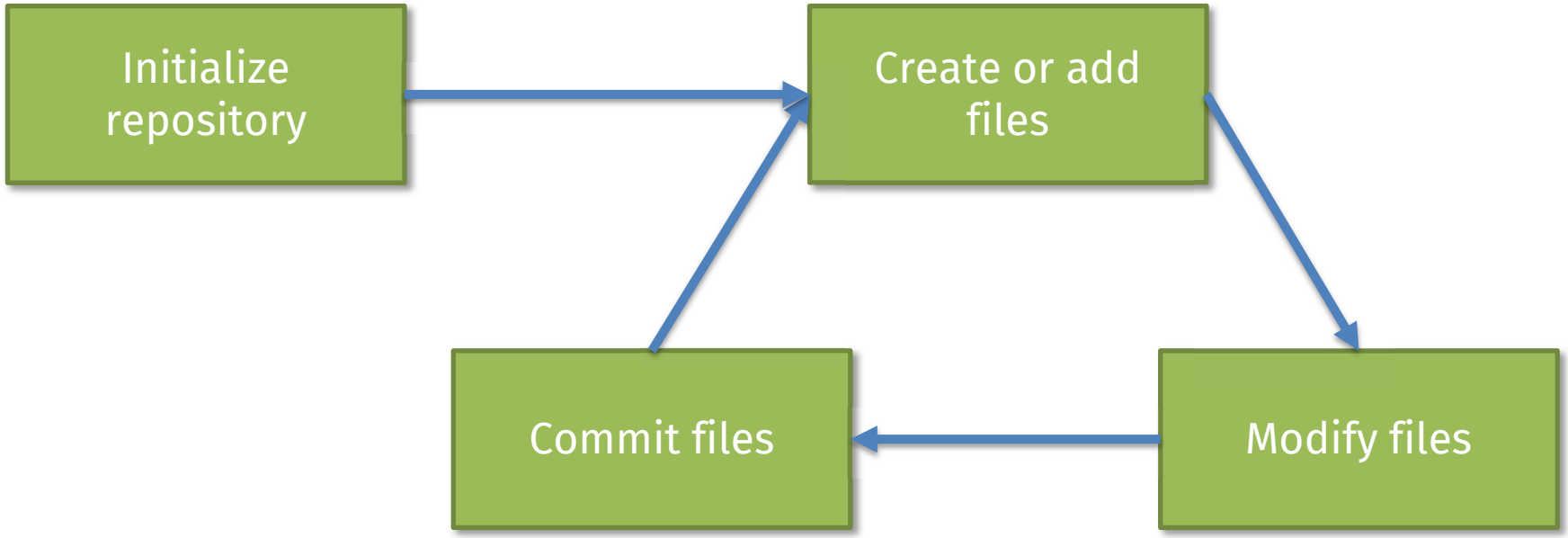
Distributed revision control system



Install & Setup

- **Linux**
  - Ubuntu : `sudo apt install git`
- **Windows, Mac**
  - download at http://git-scm.com/
- **User setup**
  - `git config --global user.name “name”`
  - `git config --global user.email “e-mail”`

Work flow on local repository

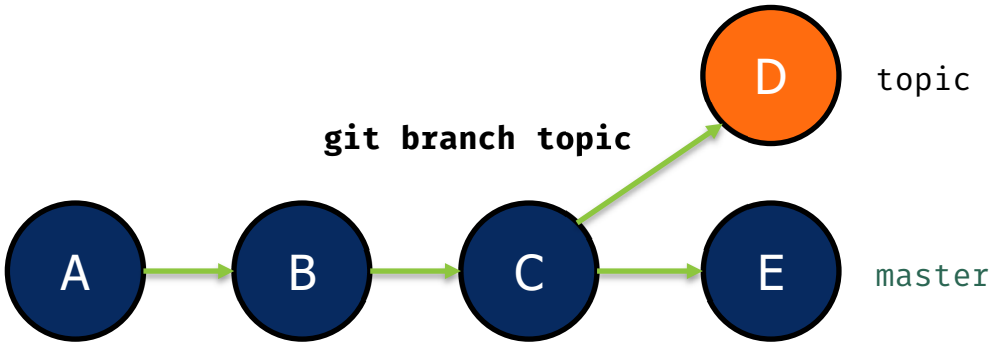


Git basic commands

- **`git init`**
  - Create an empty Git repository or reinitialize an existing one
- **`git add “filename”`**
  - Add file contents to the index
- **`git rm “filename”`**
  - Remove files from the working tree and from the index
- **`git commit`**
  - Record changes to the repository
  - options
    - `-a` : Tell the command to automatically stage files that have been m odified
    - `-m “msg”` : Use the given “*msg*” as the commit message
- **`git status`**
  - Show the working tree status

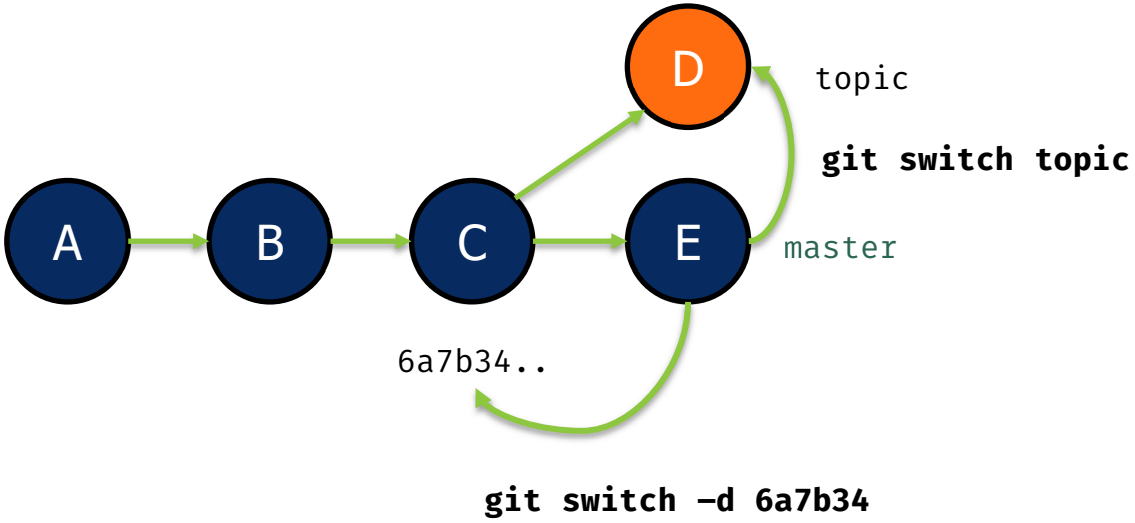
Git branch commands

- **`git branch`**
  - List, create, or delete branches
  - options
    - `[-d] "branchname"` : The name of the branch to create or delete



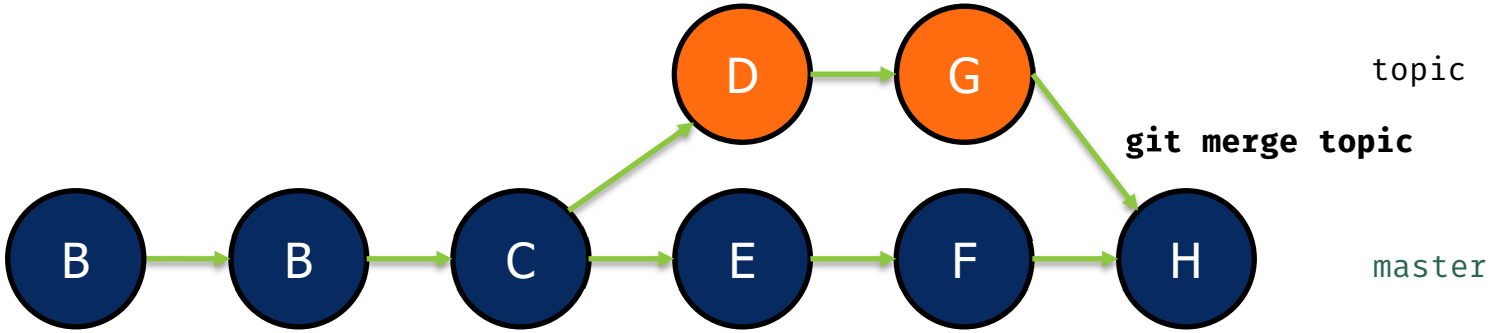
Git branch commands

- **`git switch`**
  - Switches to a different branch or commit in the working tree
  - options
    - `"branchname"` : switch to `"branchname"`
    - `-c "newbranch"` : create `"newbranch"` and switch
    - `-d "id"` : switch to a commit or tag



Git branch commands

- **git merge**
  - Join two or more development histories together
  - options
    - "`branchname`" : Reply the changes of "`branchname`" on top of current branch



Git log command

- **git log**
  - Show commit logs
  - options

`-p` : Show all changes at each commit
`--stat` : Show statistics about modified files at each commit
`--name-only` : Show only modified file name at each commit
`--relative-date` : Show commit log with relative date
`--graph` : Draw a graphical representation of the commit history

.gitignore

- **.gitignore**
  - Ignore auxiliary files such as logs, input/out data, etc
  - Generate automatically at https://www.gitignore.io/

```
# no .a files
*.a
# but do track lib.a, even ignoring .a files above
!lib.a
# ignore all files in the build/ directory
build/
```

```
git add .gitignore
git commit -m "added '.gitignore' file"
```

.gitignore

- `.gitignore`
  - Ignore auxiliary files such as logs, input/out data, etc
  - Generate automatically at https://www.gitignore.io/

```
git add .gitignore
git commit -m "added '.gitignore' file"
```

Lab Exercise 1 - Makefile

- **Create a new directory called *week3*, containing last week's plus.c, minus.c, and main.c.**
  - You can just do **`cp –R week2 week3`**
- **Write your own *Makefile* to compile the exercise**
  - In your **Makefile**,
    - Use at least one variable
    - Use at least one automatic variable
    - Make a ***clean*** rule which removes **only** the object files and executable file you have created
    - Make a rule that will create an executable called "w3"

  - Hint: check previous slides for examples of Makefile

Lab Exercise 2 - Shell

- **Write a *shell script* for scoring your "plus & minus"**
  - Name the shell script as "**score.sh**", which receives two input arguments
  - (i.e., ./score.sh A B) -> no need to error-check

  - This script follows the following process :

1) Compile your program using the *Makefile* created in Exercise 1
2) Use bash arithmetic to calculate the sum (A+B) and difference (A-B) and write those to **answer.txt**
3) Run the w3 program with the inputs and write the output to **output.txt**
4) Compare *answer.txt* and *output.txt* with ***diff*** command and store the output in **result.txt**
5) Clean object files and executable file using the *Makefile*'s clean rule

Hint 1 : Bash arithmetic is explained in slide 10.
Hint 2: *echo* and *pipe* can help you write your script (Use *man*).
Hint 3: w3 receives input through ARGV, not standard input.

Lab Exercise 2 - Shell

Example:

If your program is correct, *result.txt* should be empty

```
./score.sh 22 11
```

output.txt

```
33
11
```

answer.txt

```
33
11
```

result.txt

```

```

How to Submit

- **Materials to submit**
  - The *week3* directory should only contain **main.c**, **plus.c**, **minus.c**, **Makefile** and the **shell script**
  - Compress the *week3* directory using the tar command
    - Assuming you are in week3, `` `cd .. && tar -caf STUDENT ID_w3.tar.zst week3/` ``
  - Upload the *compressed tarball* in **iCampus** by the end of the day
  - If you are not using an AMD64 laptop (i.e. ARM64 VM, AMD64 Docker image on ARM64), please mention it as a comment.

```
" tar.vim version v32b
" Browsing tarfile /home/spl/spl/2025f/w3/2025000000_w3.tar.zst
" Select a file with cursor and press ENTER

week3/
week3/plus.c
week3/main.c
week3/Makefile
week3/minus.c
week3/score.sh
```